# Tools: Executable Entry Points

# What Is a Tool?

## What Is a Tool? I

A **tool** in the template repository is a directory under `tools/<scope>/<name>/` that packages one or more executable scripts behind a typed manifest (`tools.yaml`). Tools provide the third layer of the resource architecture: where fonds supply static data and rules supply governance constraints, tools supply *behaviour* — computations, validation runs, and agent skill invocations that projects can trigger without re-implementing the underlying logic.

## What Is a Tool? I

The tools layer deliberately mirrors the Unix philosophy of small, composable utilities that communicate through standard interfaces (Raymond 2003). Each tool declares its entrypoints (shell scripts), its invocation contract (stdin/stdout/exit-code semantics), and its capabilities (type, version, tags) in a single manifest file.
Consumers invoke tools through the `tools_invoker` module without needing to understand the tool's implementation details — a textbook application of the Facade pattern (Gamma et al. 1994).

# Tool Manifest Contract

## Tool Manifest Contract I

Every tool root must contain a `tools.yaml` manifest with the following fields:

```yaml
type: code_executor | validator | skill | agent |
renderer
description: "Human-readable description of the tool"
version: "1.0.0"
tags: [curated, exemplar, production, experimental]
creator: "org/repo"
license: "Apache-2.0"
entrypoints:
  - scripts/run.sh
  - scripts/validate.sh
```

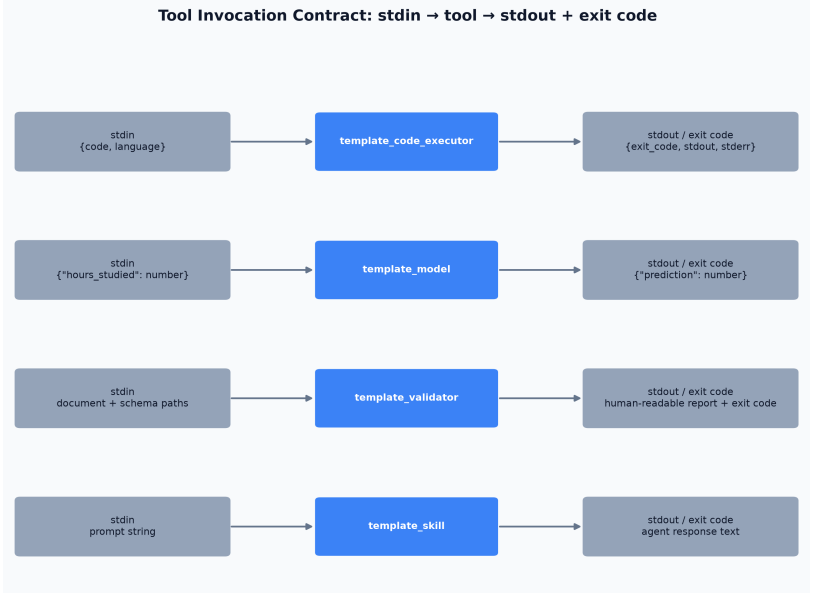
## Tool Manifest Contract I

The `type` field determines the invocation contract the consumer should expect. The `entrypoints` list names the files that must exist on disk; the `tools_invoker` module validates their presence at discovery time rather than at invocation time, making failures visible early in the pipeline rather than at runtime — a manifest naming a missing entrypoint file is rejected during discovery instead of failing later mid-invocation. fig. 4 visualises the stdin/stdout/exit-code contract for all 4 discovered template tools side by side. Note that the *shape* of stdin and stdout differs per tool while the *presence* of a well-defined contract does not — this is what makes `tools_invoker` able to discover and validate any tool generically without knowing its payload schema. A declared entrypoint that is missing blocks invocation — absent files make the tool unavailable rather than silently deferred.

## Tool Manifest Contract I

The strong-rule evaluator carries matching negative controls: `tests/rules/test_strong_rule_evaluator.py` flags missing manuscript sections, missing `src/`, and missing context keys on known-wrong fixtures.

# Tool Manifest Contract I



## Tool Manifest Contract I

Figure 4: Invocation contract for the 4 discovered template tools: stdin payload, tool behaviour, and stdout/exit-code shape.

# Discovered Template Tools

A generic code execution tool that accepts a JSON payload on standard input and returns execution results as JSON. The invocation contract is:

# template_code_executor

| Entrypoint | stdin | stdout | exit code |
|---|---|---|---|
| `scripts/run.sh` | `{"code": str, "language": str}` | `{"exit_code": int, "stdout": str, "stderr": str}` | 0 = success |
| `scripts/validate.sh` | — | Human-readable validation report | 0 = valid |

The code executor exemplifies tools that wrap a computational capability. The JSON-in/JSON-out contract makes it easily composable with pipeline orchestrators and agent frameworks.

# template_validator

A JSON Schema validation tool. It reads a target document and a schema from disk and reports validation results in human-readable form. The entrypoint `scripts/validate.sh` exits 0 when the document is valid and non-zero with a detailed error message otherwise. its `scripts/schema.json` requires `name` and `version`, and the test suite exercises both valid and invalid payloads with real subprocesses.

An agent skill invocation tool that wraps a Hermes-compatible skill definition. The entrypoint `scripts/invoke.sh` accepts a prompt string on standard input and returns the agent response as text. This tool type bridges the repository's tool architecture with external agent frameworks, demonstrating that the same manifest-and-entrypoint pattern applies equally to computational tools and AI agents.

Unlike the two tools above, `scripts/invoke.sh` requires a real `OPENAI_API_KEY` and makes a paid network call to `api.openai.com` — it is therefore never invoked by this project's tests or pipeline, by design. Offline reproducibility is a stated requirement of this exemplar (see the front-matter Reproducibility Checklist), and no CI job in this repository injects `OPENAI_API_KEY` into this project's test run. `template_code_executor`, `template_validator`, and

`template_model` are fully local and deterministic.

A pre-trained linear-regression model exemplar. It predicts a numeric target from `hours_studied` using fixed coefficients in `model_weights.json`; `scripts/predict.sh` reads `{"hours_studied": number}` and returns a JSON prediction. The manifest declares the weights file as an entrypoint, so the same discovery and existence-validation contract covers both executable scripts and model data.

# Tool Discovery Module

## Tool Discovery Module I

The src/template_pools_rules_tools/tools/tools_invoker.py module provides three public functions:

```
from template_pools_rules_tools.tools.tools_invoker
import (
    discover_tools,
    get_tool_entrypoints,
    validate_tool_scripts_exist,
)

tools = discover_tools()
# → [{"name": "template_code_executor", "manifest":
{...}}, ...]

eps = get_tool_entrypoints("template_code_executor")
# → ["scripts/run.sh", "scripts/validate.sh"]

result =
validate_tool_scripts_exist("template_code_executor")
```

# Tool Discovery Module II

```
# → {"status": "ok" | "partial" | "missing",
"missing_scripts": [...]}
```

## Tool Discovery Module I

`discover_tools()` scans `tools/templates/` and returns one `ToolEntry` per subdirectory, regardless of whether a manifest is present: a directory with a parseable `tools.yaml` gets `manifest={...}`; a directory with no manifest, or one that fails to parse, gets `manifest=None` plus a logged warning. Discovery itself never raises and never drops a directory from the result — the *interpretation* of "not a real tool yet" is left to the caller (`get_tool_entrypoints()` and `validate_tool_scripts_exist()` both return an empty/`"missing"` result for a `None` manifest), which keeps discovery and validation as separate, independently testable concerns.

## Tool Discovery Module I

`validate_tool_scripts_exist()` iterates over the manifest's `entrypoints` list and checks each path against the filesystem. It returns a structured result distinguishing between tools that are fully ready (`"ok"`), partially configured (`"partial"` — some scripts missing), and entirely absent (`"missing"`). In the current integration run, **4 tools** were discovered (fig. 5), all with valid manifests.

# Tool Discovery and Reproducibility

## Tool Discovery and Reproducibility I

The tools layer contributes to reproducibility by making the *availability* of computational capabilities explicit and checkable. A project that hard-codes a path to a tool script becomes brittle when the repository is reorganised. By contrast, a project that calls `discover_tools()` and checks `validate_tool_scripts_exist()` will detect missing entrypoints at pipeline initialisation time and report them clearly, rather than failing silently at execution time (Stodden et al. 2013). This shift from implicit to explicit dependency declaration is a key design principle of the template architecture (see fig. 1).

# Tool Composition and Failure Modes

## Tool Composition and Failure Modes I

Because every tool exposes the same discovery contract (manifest + entrypoint list), tools compose without any consumer-side special-casing. A pipeline stage that wants "any validator" can call `discover_tools()`, filter the result list on `manifest["type"] == "validator"`, and invoke whichever `template_validator`-typed tool it finds — without hard-coding `template_validator` by name. This composability comes with three failure modes that `tools_invoker` handles explicitly rather than leaving to the caller:

## Tool Composition and Failure Modes I

1. **Manifest present, entrypoint missing.** `discover_tools()` finds a parseable `tools.yaml` declaring `scripts/run.sh`, but the file does not exist. `validate_tool_scripts_exist()` surfaces this as `status="partial"` with the specific missing path in `missing_scripts`, catching the defect at discovery time instead of waiting for a caller to try executing the script.
2. **Manifest missing entirely.** A directory under `tools/templates/` exists but has no `tools.yaml`. `discover_tools()` still returns a `ToolEntry` for it (with `manifest=None`, so the caller can see it exists), but `get_tool_entrypoints()` and `validate_tool_scripts_exist()` both treat a `None` manifest as "not yet a tool," returning an empty collection or `"missing"` status without ever raising — this matters for partially-scaffolded work-in-progress tool directories created by a parallel agent.
3. **Manifest malformed YAML.** The same graceful-degradation pattern from sec. 4 applies here: a parse error is caught and logged, and the offending tool is quietly excluded from the discovery result, leaving the pipeline free to continue with everything else it found.

## Tool Composition and Failure Modes I

Each of these is a distinct, testable branch in `tests/tools/test_tools_invoker.py`, and each maps to one row of the resilience taxonomy in fig. 8.

# Execution-Proof Testing: Beyond Manifest Checking

## Execution-Proof Testing: Beyond Manifest Checking I

Everything described so far — discovery, entrypoint-existence validation, the three failure modes above — is *structural*: it confirms a tool's files are present and well-formed without ever running them. `src/template_pools_rules_tools/tools/tools_invoker.py`'s public API deliberately stays that way, because subprocess execution is exactly the kind of operation that can raise (a missing `bash`/`jq`/`python3` binary, a permission error, a timeout), and this project's readers are contracted to degrade gracefully rather than propagate exceptions (see sec. 4).

## Execution-Proof Testing: Beyond Manifest Checking I

The test suite closes this gap without weakening that contract: `tests/tools/test_tools_invoker.py` genuinely subprocess-invokes the two fully local, deterministic tools and asserts on their real output, rather than only checking that their scripts exist.

# Execution-Proof Testing: Beyond Manifest Checking I

- **`template_code_executor`**: `TestInvokeCodeExecutor` pipes `{"code": "print(2 + 2)", "language": "python"}` into the real `scripts/run.sh` and asserts the parsed JSON result has `exit_code == 0` and `"4"` in `stdout` — and, as a negative control, pipes code that calls `raise SystemExit(3)` and asserts the real `exit_code == 3` comes back.
- **`template_validator`**: `TestInvokeValidator` pipes a schema-conformant document into the real `scripts/validate.sh` and asserts `returncode == 0` and `"VALID"` in stdout; a document missing the `version` field required by the tool's own `schema.json` is asserted to return `returncode == 1` and `"INVALID"` — a genuine, schema-verified violation, not an assumed one.

Both test classes are guarded with `pytest.mark.skipif` on the real runtime dependency each script needs (`timeout`/`gtimeout` plus `jq` for the code executor; the `jsonschema` package for the validator), so a missing binary skips the test cleanly instead of failing it — on the machine this manuscript was rendered on, the code-executor tests skip (no `timeout`/`gtimeout` on this macOS host) while the validator tests run for real, which is itself a demonstration of the skip-guard working as intended rather than masking a failure.